

实验四：微信小程序开发实验报告

学生姓名：费韵

学号：2023210591

实验日期：2026 年 4 月 18 日

一、实验目的

- 掌握微信小程序的基础开发流程，理解 WXML、WXSS、JavaScript 在小程序开发中的分工与协作。
- 实现“智慧校园通知”小程序的通知列表展示、详情页跳转及本地存储功能，理解页面路由与参数传递机制。
- 体验 AI 编程工具（TRAE）在代码生成与调试中的辅助作用，提升开发效率。
- 完成小程序真机预览与体验版发布，了解小程序测试与部署的基本流程。

二、实验环境

- 硬件环境：Windows 11 操作系统、手机（微信小程序真机预览）
- 软件环境：微信开发者工具、TRAE AI 编程工具
- 开发语言：WXML、WXSS、JavaScript

三、实验内容与步骤

本次实验围绕“智慧校园通知”小程序展开，核心功能包括通知列表展示、详情页跳转、已读状态本地存储，完整开发流程如下：

（一）需求分析与设计

- 页面流程设计**：规划了“通知列表页 → 通知详情页”的基础流程，列表页点击通知项通过页面路由跳转至详情页，同时通过本地存储记录用户的已读状态。
- 数据模型定义**：设计通知数据结构，包含 id（通知唯一标识）、title（通知标题）、author（发布部门）、time（发布时间）、content（通知详情）、isRead（已读状态）等字段，为后续开发提供数据支撑。

（二）项目创建与基础配置

1. 打开微信开发者工具，新建小程序项目，使用测试号完成项目初始化，选择 JavaScript 基础模板。
2. 配置 app.json 文件，注册 pages/index/index（通知列表页）和 pages/detail/detail（通知详情页）两个页面路径，设置全局导航栏标题。

（三）通知列表页开发

1. **WXML 结构实现：**使用 wx:for 循环渲染通知列表，为每个通知项绑定 bindtap 点击事件，通过 data-id 传递通知 ID，实现跳转准备。同时通过 {{item.isRead ? 'read' : ''}} 动态绑定已读样式，区分已读/未读通知。
2. **WXSS 样式编写：**设置列表项的边距、边框、背景色，定义 .read 类的透明度样式，让已读通知呈现弱化效果，提升用户体验。
3. **JS 逻辑实现：**
 - onLoad 生命周期中调用 loadNews 方法加载模拟通知数据。
 - 读取本地存储的 readIds 数组，为通知数据标记已读状态。
 - 实现 goToDetail 方法，通过 wx.navigateTo 跳转至详情页并传递通知 ID 参数。
 - 配置下拉刷新功能，在 onPullDownRefresh 中重新加载通知数据。

（四）通知详情页开发

1. **WXML 结构实现：**接收列表页传递的通知数据，展示通知标题、发布部门、发布时间及详情内容，排版符合校园通知的阅读习惯。
2. **JS 逻辑实现：**
 - onLoad 生命周期中获取列表页传递的通知 ID，通过模拟数据匹配通知详情。
 - 实现 markAsRead 方法，将当前通知 ID 存入本地存储的 readIds 数组，标记通知为已读。

（五）本地存储功能实现

1. 列表页加载时，通过 wx.getStorageSync('readIds') 读取本地已读通知 ID，为通知数据标记状态。
2. 详情页中，通过 wx.setStorageSync('readIds', readIds) 更新本地存储，记录用户的已读操作，实现数据持久化。

（六）测试与部署

1. **真机预览测试：**使用微信开发者工具编译项目，扫描二维码在手机上预览，测试列表渲染、页面跳转、已读状态更新等功能，验证与预期效果一致。
2. **功能验证：**测试通知列表的展示效果、详情页内容完整性、已读状态标记与存储功能，确认所有交互逻辑正常。
3. **体验版发布：**在微信开发者工具中上传项目代码，填写版本信息，生成体验版二维码，完成小程序的初步部署。

四、实验结果

本次实验成功完成了“智慧校园通知”小程序的开发，核心功能均实现并通过测试，具体结果如下：

1. **通知列表页：**成功渲染校园通知列表，清晰展示通知标题、发布部门与时间，已读通知呈现弱化样式，下拉刷新功能正常。
2. **详情页跳转：**点击列表中的通知项可正常跳转至详情页，完整展示通知标题、发布信息与详情内容，跳转参数传递无误。
3. **本地存储功能：**点击通知后，列表页对应通知自动标记为已读，关闭小程序后重新打开，已读状态保持不变，数据持久化功能正常。
4. **真机预览效果：**在手机端可正常预览小程序，界面适配良好，交互流畅，所有功能均符合预期设计。



智慧校园通知

已连接

展开

关于期末考试安排的通知

教务处

2026-04-15

图书馆开放时间调整

图书馆

2026-04-15

校园网络维护通知

信息中心

2026-04-15

关于举办科技创新大赛的通知

科技处

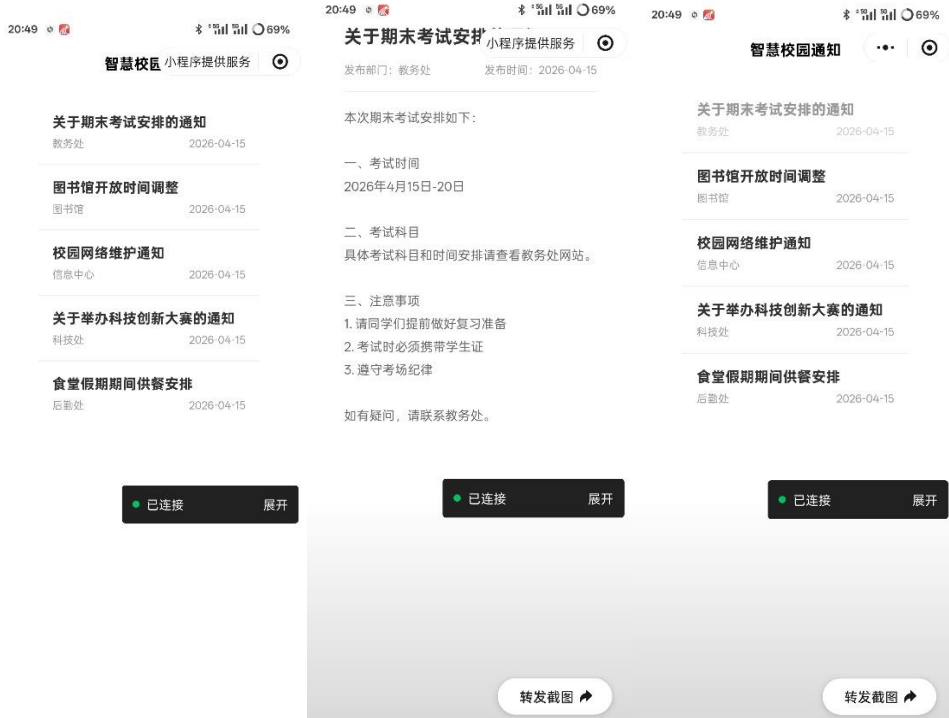
2026-04-15

食堂假期期间供餐安排

后勤处

2026-04-15

测试截图-下拉刷新



测试截图-灰色表示已读

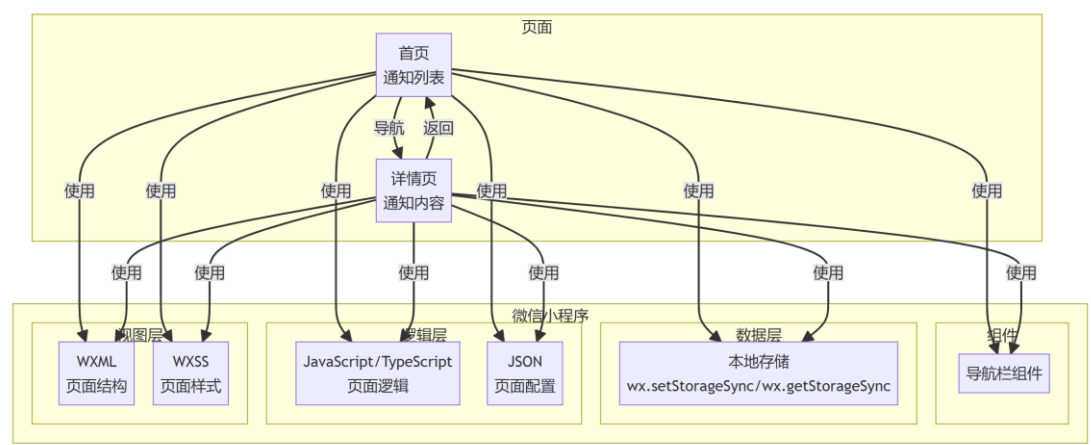
五、项目图表

本微信小程序采用三层架构：视图层（WXML/WXSS）、逻辑层（JavaScript/TypeScript）和数据层

（一）架构图

```
miniprogram/
├─ pages/
│  └─ index/      # 通知列表页
│     │  └─ index.wxml # 页面结构
│     │  └─ index.ts   # 页面逻辑
│     │  └─ index.wxss # 页面样式
│     │  └─ index.json # 页面配置
│     └─ detail/    # 详情页
│        │  └─ detail.wxml # 页面结构
│        │  └─ detail.ts   # 页面逻辑
│        │  └─ detail.wxss # 页面样式
│        │  └─ detail.json # 页面配置
├─ components/
│  └─ navigation-bar/ # 导航栏组件
│     │  └─ navigation-bar.wxml
│     │  └─ navigation-bar.ts
│     │  └─ navigation-bar.wxss
│     │  └─ navigation-bar.json
├─ app.json          # 全局配置
├─ app.ts             # 应用逻辑
└─ app.wxss           # 全局样式
```

1. 微信小程序架构图



核心文件说明：

app.json: 全局配置文件，定义小程序的页面路径、窗口样式、导航栏等。

app.ts: 应用逻辑文件，处理应用的生命周期、全局数据等。

pages/index/index.wxml: 通知列表页的结构文件，使用 WXML 语法定义页面布局。

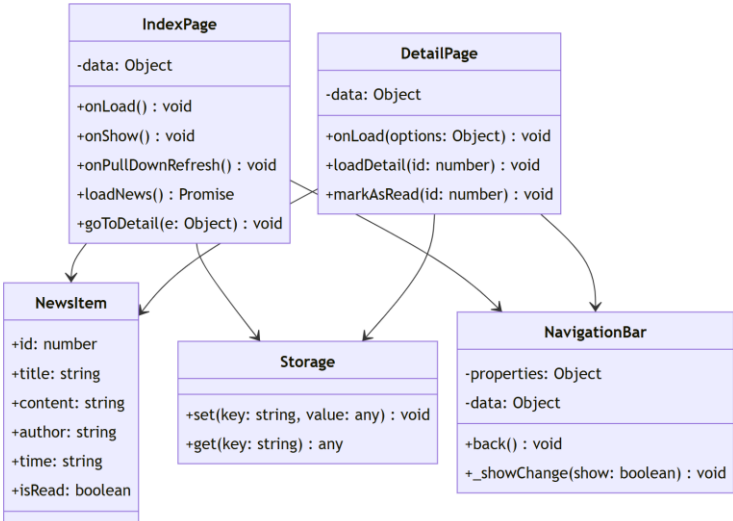
pages/index/index.ts: 通知列表页的逻辑文件，处理数据加载、事件处理等。

pages/detail/detail.wxml: 详情页的结构文件，展示通知的详细内容。

pages/detail/detail.ts: 详情页的逻辑文件，处理参数接收、内容展示等。

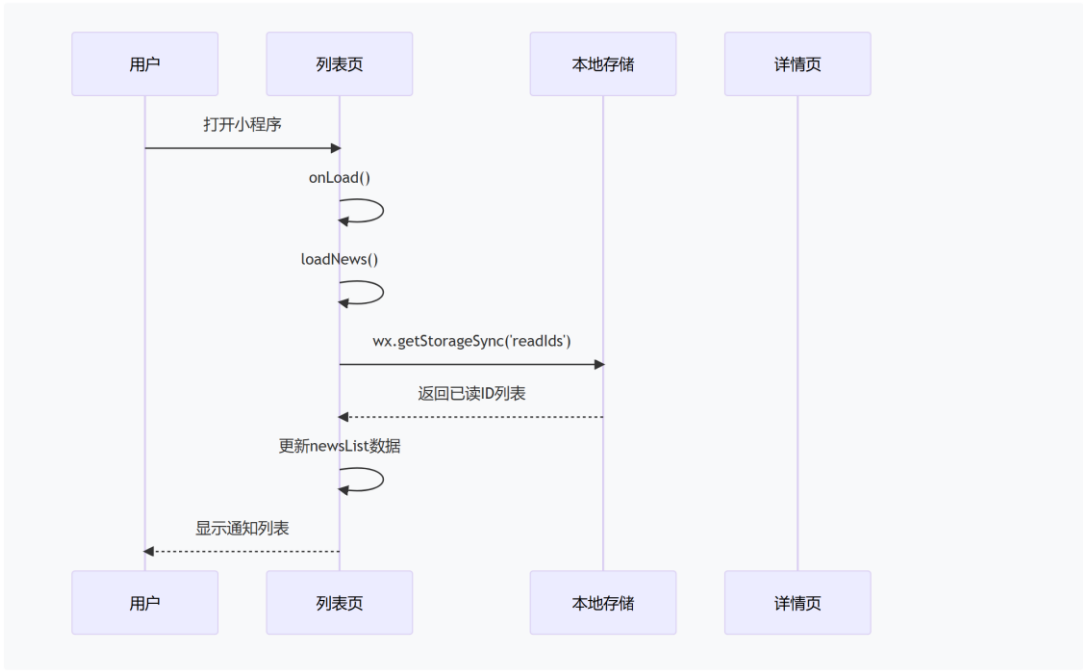
(二) 类图

2. 微信小程序类图

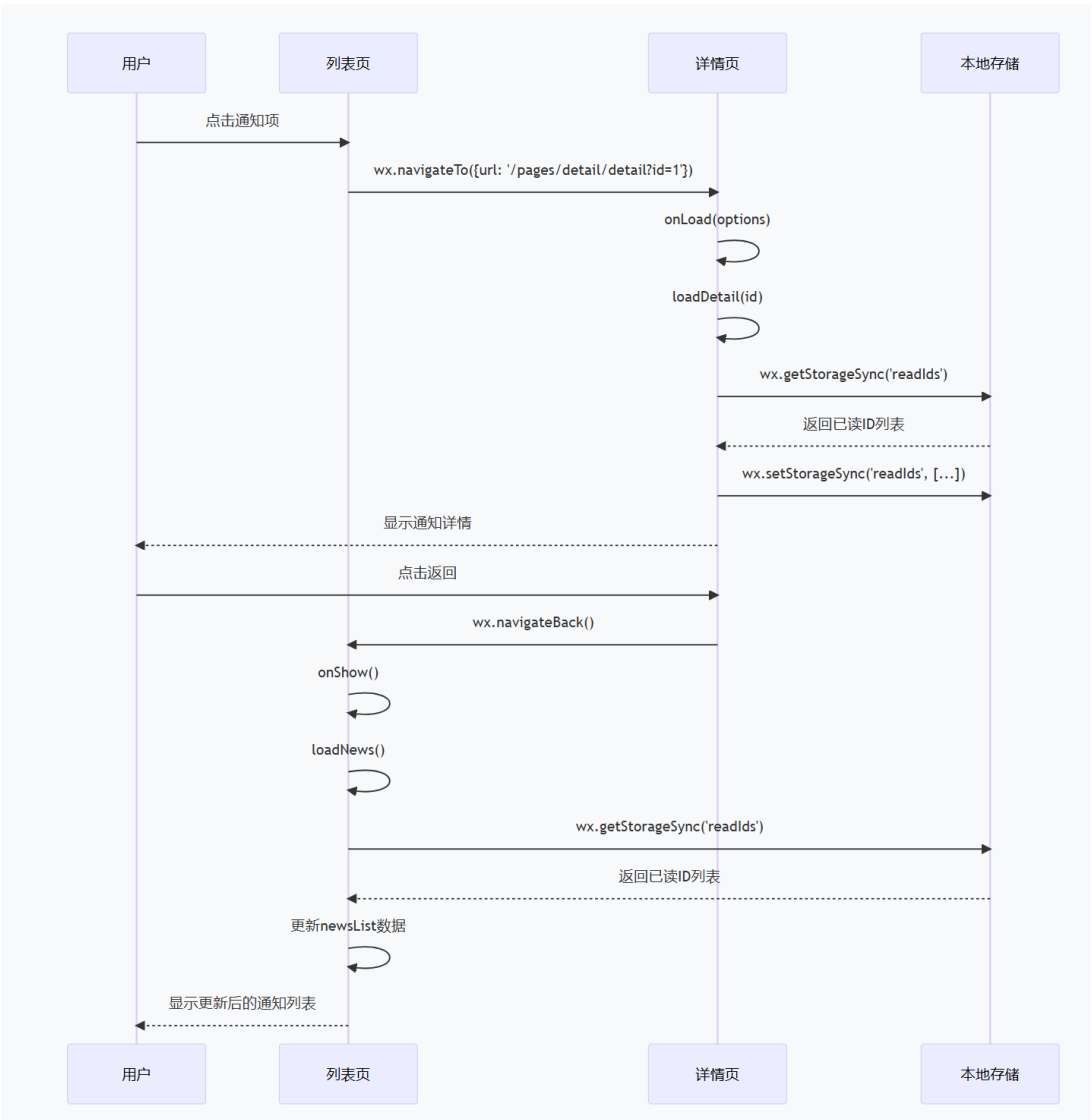


(三) 流程图

3.1 通知列表加载流程



3.2 查看通知详情流程



(4) 功能说明

4.1 通知列表页

通知列表页主要功能：

- 展示通知列表，包含标题、发布部门和时间
- 已读通知显示灰色字体
- 支持下拉刷新功能
- 点击通知项跳转到详情页

4.2 详情页

详情页主要功能：

- 接收并显示通知详情
- 自动标记通知为已读
- 支持返回列表页

4.3 本地存储

使用微信小程序的本地存储 API 实现已读状态的持久化：

- 使用 `wx.setStorageSync('readIds', readIds)` 保存已读通知 ID
- 使用 `wx.getStorageSync('readIds')` 读取已读通知 ID
- 已读状态在列表页和详情页之间同步

六、实验总结与思考

（一）实验总结

本次实验完整经历了微信小程序从需求分析、AI 辅助编码、测试验证到部署的全流程，成功开发了具备基础功能的校园通知小程序。通过实验，我掌握了微信小程序的页面开发、路由跳转、本地存储等核心技术，理解了 WXML、WXSS、JavaScript 三者的协作方式，同时体验了 AI 编程工具在代码生成中的辅助作用，有效提升了开发效率。实验中，通知列表的循环渲染、参数传递、已读状态的持久化存储等关键环节均成功实现，验证了设计方案的可行性。

（二）问题与解决

1. **问题 1：**列表页点击通知后，详情页无法正确加载对应通知内容。

解决方法：检查发现 `goToDetail` 方法中 `data-id` 的参数传递存在格式错误，修正参数传递逻辑后，详情页可正确匹配通知 ID 并加载数据。

2. **问题 2：**已读状态在页面刷新后无法保持。

解决方法：排查发现本地存储的 `readIds` 数组未正确更新，调整 `markAsRead` 方法中存储逻辑，确保数组更新后调用 `wx.setStorageSync`，实现数据持久化。

（三）课后思考

1. 小程序相比原生 App，开发成本更低、无需下载安装、更新便捷，适合校园这类轻量化场景的快速开发与推广；但在复杂功能实现、系统权限调用方面存在一定限制。
2. AI 工具在代码生成、调试辅助中可快速生成基础代码框架，减少重复劳动，但仍需开发者手动优化逻辑、处理边界问题，合理利用 AI 工具可显著提升开发效率。
3. 小程序在校园场景中可拓展更多创新应用，如校园活动报名、课程表查询、校园服务预约等，结合本地存储与云开发，可实现更丰富的师生服务功能。

七、实验心得

通过本次实验，我对微信小程序开发有了系统的认知，从基础语法到功能实现，逐步掌握了小程序开发的核心流程。AI 工具的辅助让我在代码编写中少走了很多弯路，也让我意识到合理利用工具对提升开发效率的重要性。同时，在调试与测试过程中，我深刻体会到了前端开发中细节处理的重要性，一个参数传递的错误就可能导致整个功能失效。本次实验不仅提升了我的技术能力，也培养了我分析问题、解决问题的能力，为后续校园场景小程序的开发打下了坚实基础。